



INSTITUTO POLITÉCNICO NACIONAL



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

SISTEMAS EN CHIP

Reporte de la práctica 2

“Contador Ascendente y Descendente”

Grupo: 6CM1

Integrantes

- García Escamilla Bryan Alexis
- Meléndez Macedonio Rodrigo

Profesor

Aguilar Sánchez Fernando

Fecha de entrega: 13 de febrero de 2025

INTRODUCCIÓN

Contador

Un contador es un circuito secuencial construido a partir de flip-flops conectados en cascada. Su función principal es seguir una secuencia de estados binarios en respuesta a una señal de reloj (clock). Básicamente, cuenta cuántos pulsos han ocurrido en su entrada, lo cual es fundamental para relojes digitales, temporizadores y procesos de automatización.

Clasificación:

- **Contadores Asíncronos:**
 - La señal de reloj solo entra al primer flip-flop. El cambio del primer flip-flop dispara al segundo, y así sucesivamente.
 - Desventaja: Tienen un "retraso de propagación". Como cada etapa espera a la anterior, a altas frecuencias pueden ocurrir errores de lectura.
- **Contadores Síncronos:**
 - Todos los flip-flops reciben la señal de reloj al mismo tiempo.
 - Ventaja: Son mucho más rápidos y precisos porque todos los bits cambian simultáneamente. Es el tipo de contador que encuentras dentro del ATmega8535.

Para entender cualquier contador, se deben de conocer estos dos términos:

- **Módulo (MOD):** Es el número de estados distintos por los que pasa el contador antes de volver a cero.
- **Dirección de conteo:**
 - **Up-counter:** Cuenta hacia arriba (0, 1, 2...).
 - **Down-counter:** Cuenta hacia atrás (9, 8, 7...).
 - **Up/Down:** Puede cambiar de dirección según una señal de control.

Contadores en microcontroladores.

En el área de sistemas embebidos, los contadores no suelen ser chips externos, sino periféricos internos llamados Timers/Counters.

- **Modo Contador:** El chip cuenta eventos externos (como personas cruzando un sensor óptico) que entran por un pin específico (como el pin T1).
- **Modo Temporizador (Timer):** El chip cuenta los ciclos de su propio reloj interno. Como sabemos que el reloj es constante (ej. 16 MHz), contar pulsos es equivalente a medir el tiempo.

- **Prescaler:** Es un circuito que "divide" la frecuencia del reloj antes de que llegue al contador, permitiendo medir tiempos más largos sin que el contador se desborde tan rápido.

Tipos de contadores.

- **Contador BCD (Década):** Un contador que se reinicia al llegar a 10 (cuenta de 0 a 9). Es el que se usa para manejar displays de 7 segmentos en relojes.
- **Contador de Anillo:** Solo un bit está en "1" y va rotando por todas las posiciones.
- **Contador Johnson:** Similar al de anillo, pero ofrece el doble de estados usando la misma cantidad de flip-flops.

DESARROLLO

Circuito en hecho en Proteus

Para el desarrollo de la práctica 2, el circuito a armar es el siguiente:

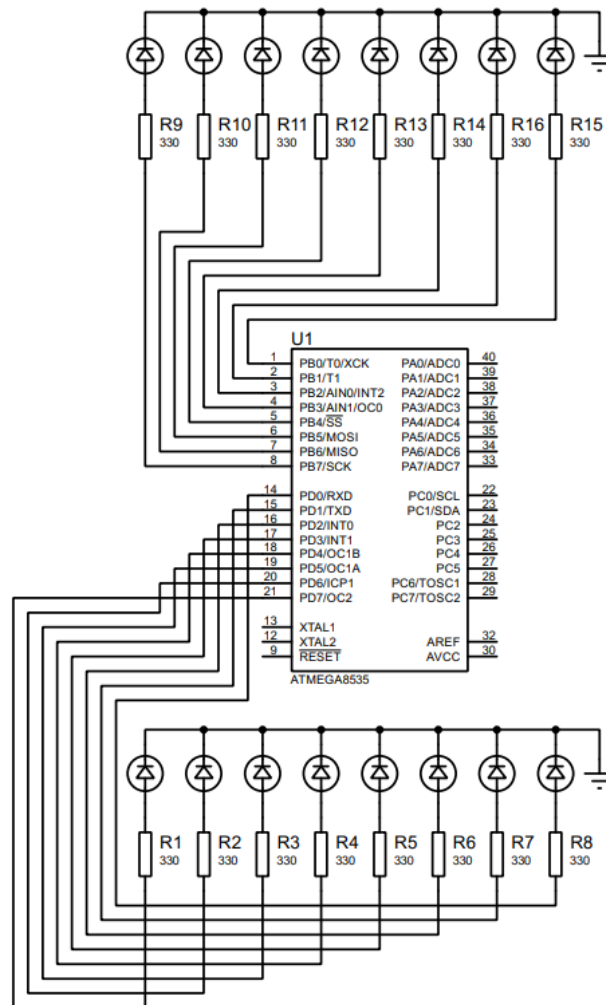


Imagen 1. Circuito creado en Proteus.

Configuración previa al código

De acuerdo con la imagen del circuito, ambos registros B y D se configuran como de salida, con un valor de 0xFF para rregistro B y 0x00 para el registro D.

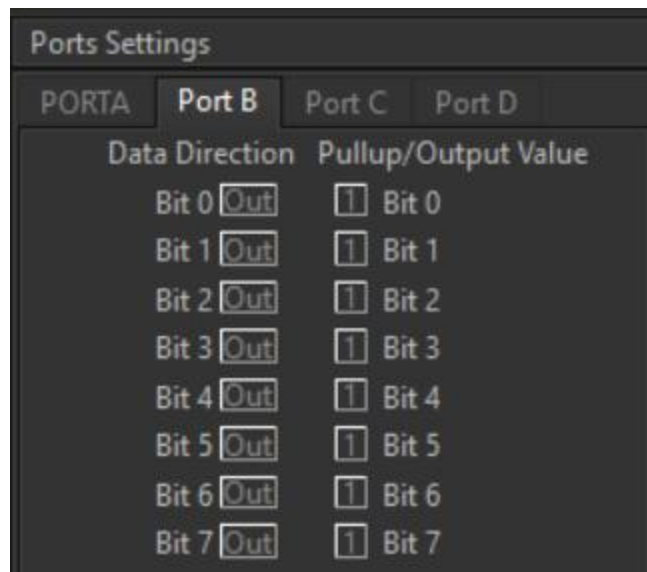


Imagen 2. Configuración del registro B.

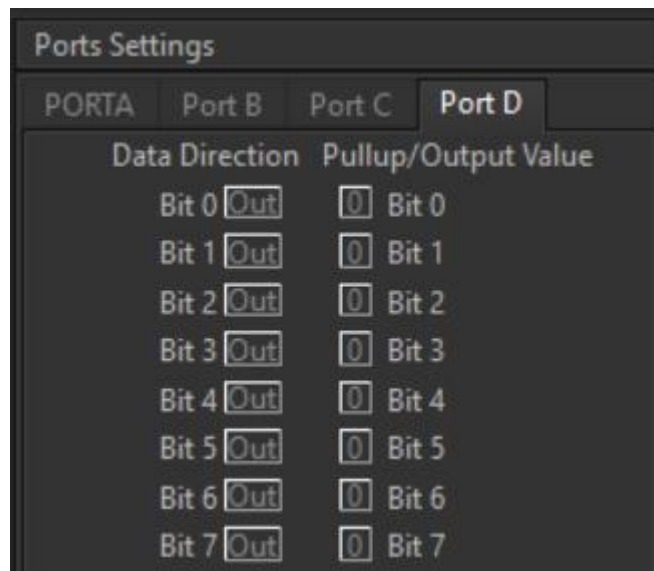


Imagen 3. Configuración del registro D.

Código del circuito de CodeVision

[Aquí se ponen las capturas del código]

```

1  /*****
2  This program was created by the CodeWizardAVR V4.06a
3  Automatic Program Generator
4      Copyright 1998-2025 Pavel Haiduc, HP InfoTech S.R.L.
5  http://www.hpinfotech.ro
6
7  Project : Pr  ctica 2 'Contador Ascendente y Descendente'
8  Version : 1.0
9  Date    : 07/02/2026
10 Author  : Garc  a Escamilla Bryan Alexis
11 Company :
12 Comments:
13
14
15 Chip type      : ATmega8535
16 Program type   : Application
17 AVR Core Clock frequency: 1.000000 MHz
18 Memory model   : Small
19 External RAM size : 0
20 Data Stack size : 128
21 *****/
22
23 // I/O Registers definitions
24 #include <mega8535.h>
25 #include <delay.h>

```

```

27 // Declare your global variables here
28
29 void main(void) {
30     // Declare your local variables here
31
32     // Input/Output Ports initialization
33     // Port A initialization
34     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
35     DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
36     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
37     PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);
38
39     // Port B initialization
40     // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
41     DDRB=(1<<ddb7) | (1<<ddb6) | (1<<ddb5) | (1<<ddb4) | (1<<ddb3) | (1<<ddb2) | (1<<ddb1) | (1<<ddb0);
42     // State: Bit7=1 Bit6=1 Bit5=1 Bit4=1 Bit3=1 Bit2=1 Bit1=1 Bit0=1
43     PORTB=(1<<PORTB7) | (1<<PORTB6) | (1<<PORTB5) | (1<<PORTB4) | (1<<PORTB3) | (1<<PORTB2) | (1<<PORTB1) | (1<<PORTB0);
44
45     // Port C initialization
46     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
47     DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
48     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
49     PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);

```

```

51     // Port D initialization
52     // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
53     DDRD=(1<<ddd7) | (1<<ddd6) | (1<<ddd5) | (1<<ddd4) | (1<<ddd3) | (1<<ddd2) | (1<<ddd1) | (1<<ddd0);
54     // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
55     PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);
56
57     // Timer/Counter 0 initialization
58     // Clock source: System Clock
59     // Clock value: Timer 0 Stopped
60     // Mode: Normal top=0xFF
61     // OC0 output: Disconnected
62     TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) | (0<<CS01) | (0<<CS00);
63     TCNT0=0x00;
64     OCR0=0x00;

```

```

66 // Timer/Counter 1 initialization
67 // Clock source: System Clock
68 // Clock value: Timer1 Stopped
69 // Mode: Normal top=0xFFFF
70 // OC1A output: Disconnected
71 // OC1B output: Disconnected
72 // Noise Canceler: Off
73 // Input Capture on Falling Edge
74 // Timer1 Overflow Interrupt: Off
75 // Input Capture Interrupt: Off
76 // Compare A Match Interrupt: Off
77 // Compare B Match Interrupt: Off
78 TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
79 TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) | (0<<CS10);
80 TCNT1H=0x00;
81 TCNT1L=0x00;
82 ICR1H=0x00;
83 ICR1L=0x00;
84 OCR1AH=0x00;
85 OCR1AL=0x00;
86 OCR1BH=0x00;
87 OCR1BL=0x00;

```

```

89 // Timer/Counter 2 initialization
90 // Clock source: System Clock
91 // Clock value: Timer2 Stopped
92 // Mode: Normal top=0xFF
93 // OC2 output: Disconnected
94 ASSR=0<<AS2;
95 TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) | (0<<CS21) | (0<<CS20);
96 TCNT2=0x00;
97 OCR2=0x00;
98
99 // Timer(s)/Counter(s) Interrupt(s) initialization
100 TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);
101
102 // External Interrupt(s) initialization
103 // INT0: off
104 // INT1: off
105 // INT2: off
106 MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
107 MCUCSR=(0<<ISC2);
108
109 // USART initialization
110 // USART disabled
111 UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) | (0<<UCSZ2) | (0<<RXB8) | (0<<TXB8);

```

```

113 // Analog Comparator initialization
114 // Analog Comparator: Off
115 // The Analog Comparator's positive input is
116 // connected to the AIN0 pin
117 // The Analog Comparator's negative input is
118 // connected to the AIN1 pin
119 ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1) | (0<<ACIS0);
120 SFIOR=(0<<ACME);
121
122 // ADC initialization
123 // ADC disabled
124 ADCSRA=(0<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) | (0<<ADPS1) | (0<<ADPS0);
125
126 // SPI initialization
127 // SPI disabled
128 SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1) | (0<<SPR0);
129
130 // TWI initialization
131 // TWI disabled
132 TWCR=(0<<TWIEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);

```



```
134 while (1) {  
135     // Place your code here  
136     PORTB++; // ascendente  
137     PORTD--; // descendente  
138     delay_ms(300);  
139 }  
140 }
```

Circuito físico

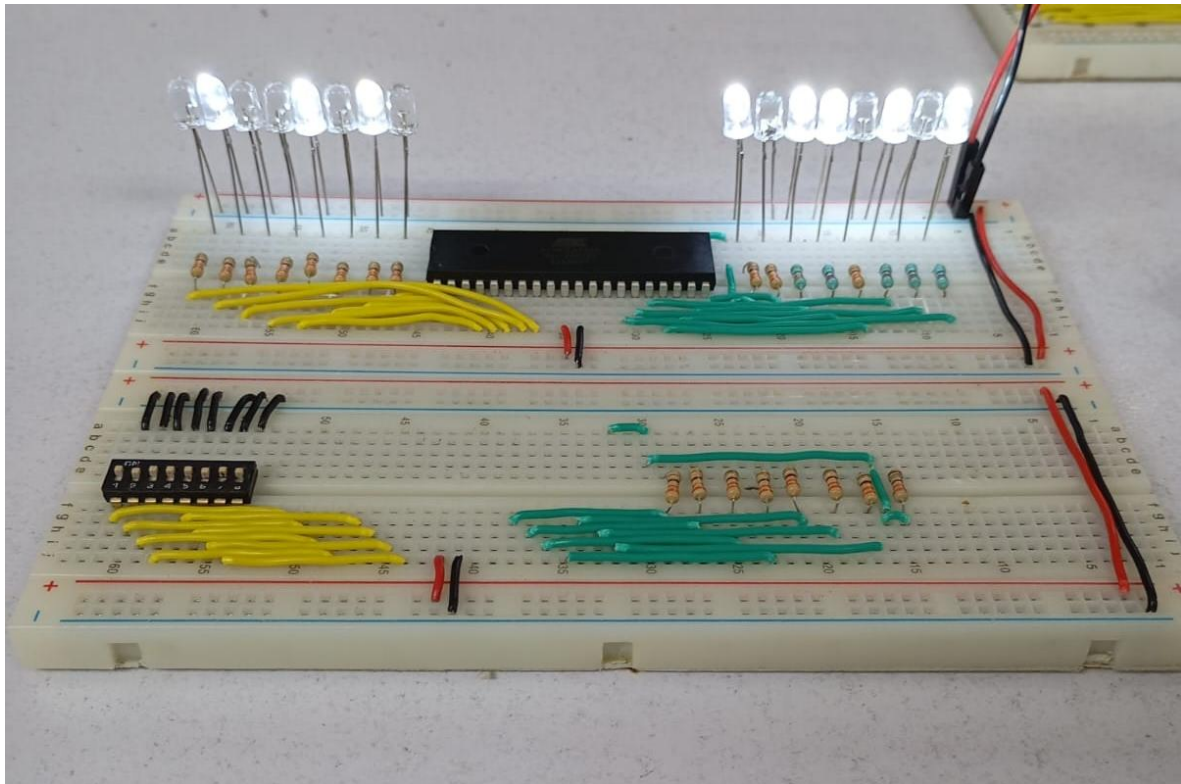


Imagen 3. Circuito físico de la práctica 2.

CONCLUSIÓN

- **García Escamilla Bryan Alexis**

En esta práctica la novedad fue el poder incrementar y decrementar el valor del registro B y D de la ATMEGA8535 de forma binaria, configurándolos previamente como de salida. Para ello se usa '++' o '--' para incrementar y decrementar respectivamente (PORTB++ y PORTD--) el valor del registro, similar a como se hace en lenguaje C.

Ambos puertos se terminan reiniciando automáticamente una vez que se llega al valor máximo en cada caso de forma indefinida, puesto que estas instrucciones se encuentran dentro de un bucle `while(1)` infinito.

Por último, por cada incremento y decremento en los registros, se coloco un delay de 300 milisegundos haciendo uso de la función `delay_ms()` de la biblioteca `delay.h` que se incluye en el código del programa, esto para que se logre apreciar adecuadamente las operaciones sobre ambos registros.

- **Meléndez Macedonio Rodrigo**

En esta práctica desarrollamos dos contadores, uno ascendente y otro descendente, para ambos casos, se utilizó un ciclo “while” que repite la numeración en binario de 8 bits, en el caso del contador ascendente fuimos incrementado el valor del puerto B con “++” y en el caso del contador descendente disminuimos el valor del puerto A con “--”. Por otro lado, para que el ojo humano pudiera visualizar el cambio de los valores en los LED’s se utilizó una función “delay_ms” con 300 milisegundos de retraso y básicamente en eso consistió el código en C. No tuvimos problemas en programar el ATMEGA8535 y todo salió de acuerdo con lo esperado.

BIBLIOGRAFÍA

- Tocci, R. J., & Widmer, N. S. (2003). *Sistemas Digitales: Principios y Aplicaciones*. Prentice Hall.